

Ariko Yahya Setyawan
Portofolio

Klasifikasi Data Tabular Menggunakan Deep Learning

Studi Kasus: Prediksi Diabetes Pasien Berdasarkan Data Medis & Fisik
Dataset: Diabetes Dataset (Vanderbilt / John Schorling) — Kaggle.com
Platform: Google Colaboratory | Framework: TensorFlow / Keras

Dibuat menggunakan Jupyter Notebook pada Google Colaboratory

1. Judul dan Tujuan Proyek

Judul: Klasifikasi Data Tabular Menggunakan Deep Learning untuk Prediksi Diabetes

Tujuan Proyek:

- Membangun model klasifikasi berbasis Deep Learning (Artificial Neural Network) menggunakan TensorFlow/Keras.
- Mengklasifikasikan pasien ke dalam dua kategori: **Diabetes** dan **Tidak Diabetes**, berdasarkan data medis dan fisik (kolesterol, gula darah, tekanan darah, usia, berat/tinggi badan, dll).
- Melakukan proses lengkap data science: eksplorasi data (EDA), praproses data, pembangunan arsitektur model, pelatihan, hingga evaluasi performa model.
- Menghasilkan model yang dapat dijadikan dasar untuk skrining awal risiko diabetes pada pasien.

Sumber Data: Dataset diunduh dari Kaggle.com, berisi 403 data pasien dengan 19 kolom atribut medis dan demografis. Label target ("Outcome") diturunkan dari nilai *glycosolated hemoglobin* (glyhb) dengan standar klinis: $\text{glyhb} \geq 7.0$ dikategorikan Diabetes, dan $\text{glyhb} < 7.0$ dikategorikan Tidak Diabetes.

2. Langkah-Langkah Pengerjaan

Proyek ini dikerjakan dalam 6 tahapan utama menggunakan Jupyter Notebook (Google Colaboratory): (1) Upload dan pemuatan dataset, (2) Eksplorasi Data (EDA), (3) Praproses data, (4) Membangun arsitektur Deep Learning, (5) Melatih model, dan (6) Evaluasi model. Berikut rincian tiap tahapan beserta kode program yang digunakan.

2.1 Upload dan Memuat Dataset

Dataset diunggah langsung ke Google Colab, kemudian dibaca menggunakan pustaka pandas.

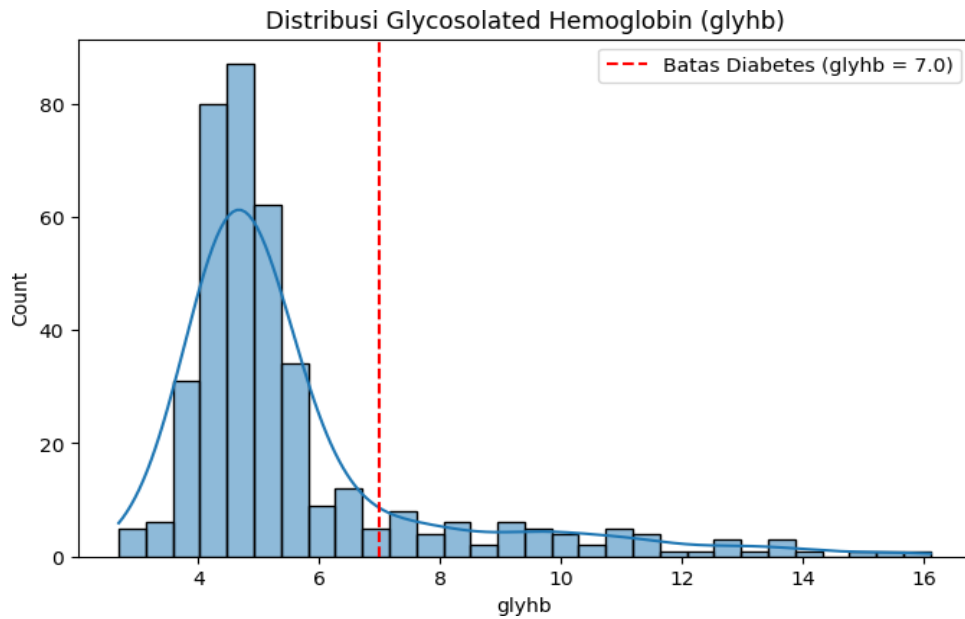
```
df = pd.read_csv(csv_path)
df.head()
```

Ukuran data: 403 baris $\times$ 19 kolom, terdiri dari data numerik (kolesterol, glukosa, tekanan darah, dsb.) dan kategorikal (gender, location, frame). Beberapa kolom memiliki missing value, khususnya *bp.2s* dan *bp.2d* yang lebih dari 60% datanya kosong.

2.2 Eksplorasi Data (EDA)

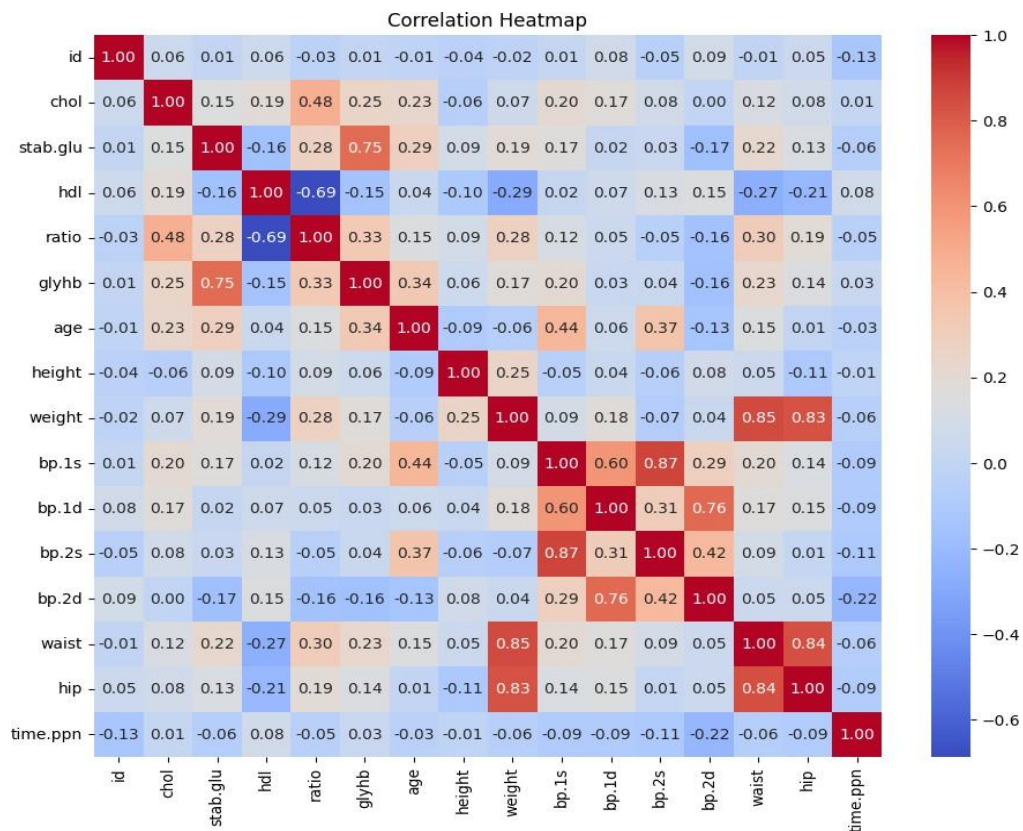
Eksplorasi dilakukan untuk memahami distribusi data dan hubungan antar variabel sebelum membangun model. Distribusi nilai glyhb digunakan sebagai dasar penentuan label target, dengan batas klinis 7.0 sebagai pemisah kelas Diabetes / Tidak Diabetes.

```
# Distribusi variabel glyhb (dasar penentuan target)
plt.figure(figsize=(8, 5))
sns.histplot(df['glyhb'].dropna(), bins=30, kde=True)
plt.axvline(7.0, color='red', linestyle='--', label='Batas Diabetes (glyhb = 7.0)')
plt.title('Distribusi Glycosolated Hemoglobin (glyhb)')
plt.xlabel('glyhb')
plt.legend()
plt.show()
```



Gambar 1. Distribusi nilai Glycosolated Hemoglobin (glyhb) pada seluruh pasien. Garis merah putus-putus menandai batas klinis (7.0) yang memisahkan kelas Diabetes dan Tidak Diabetes.

Selanjutnya, hubungan antar variabel numerik dianalisis menggunakan correlation heatmap untuk mengidentifikasi fitur-fitur yang paling berkaitan.



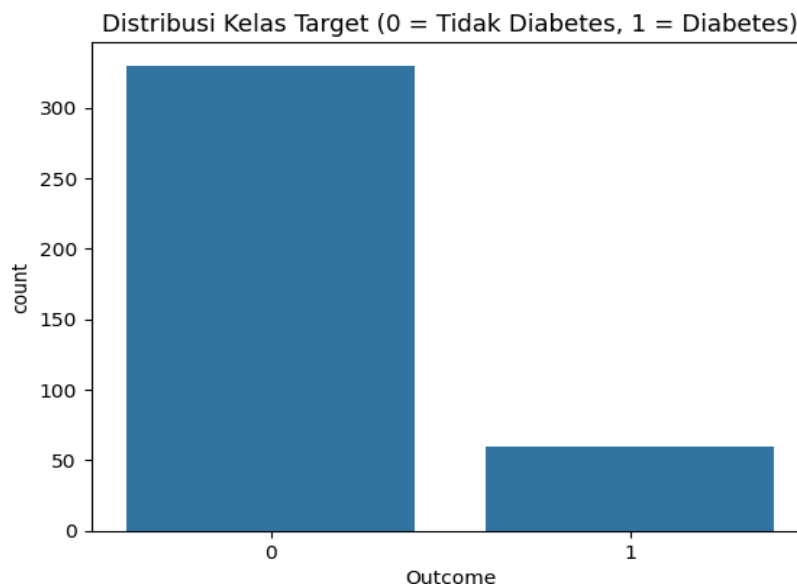
Gambar 2. Correlation heatmap antar fitur numerik dalam dataset.

2.3 Praproses Data (Preprocessing)

Tahap praproses meliputi: (1) pembuatan label target dari kolom glyhb, (2) pembuangan kolom yang tidak relevan atau memiliki terlalu banyak missing value, (3) encoding variabel kategorikal, (4) imputasi missing value dengan median, serta (5) pembagian data latih/uji dan standarisasi fitur.

```
# 1. Buat target dari glyhb, buang baris tanpa glyhb
df = df.dropna(subset=['glyhb']).copy()
df['Outcome'] = (df['glyhb'] >= 7.0).astype(int)

print(df['Outcome'].value_counts())
sns.countplot(x='Outcome', data=df)
plt.title('Distribusi Kelas Target (0 = Tidak Diabetes, 1 = Diabetes)')
plt.show()
```



Gambar 3. Distribusi kelas target hasil konversi dari glyhb (0 = Tidak Diabetes, 1 = Diabetes). Terlihat data tidak seimbang: mayoritas pasien berada pada kelas Tidak Diabetes.

```
# 2 & 3. Buang kolom yang tidak dipakai
df = df.drop(columns=['id', 'glyhb', 'bp.2s', 'bp.2d'])

# 4. Encoding kolom kategorikal
df = pd.get_dummies(df, columns=['gender', 'location', 'frame'], drop_first=True)

df.head()

# 6. Split fitur (X) dan target (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Data latih:", X_train.shape)
print("Data uji :", X_test.shape)
```

2.4 Membangun Arsitektur Deep Learning

Model yang digunakan adalah Artificial Neural Network (ANN) dengan beberapa hidden layer berfungsi aktivasi ReLU dan lapisan Dropout untuk mencegah overfitting. Karena kasus ini adalah klasifikasi biner, lapisan output menggunakan fungsi aktivasi sigmoid.

```

model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(16, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(8, activation='relu'),
    layers.Dense(1, activation='sigmoid') # output biner
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

2.5 Melatih Model

Karena distribusi kelas target tidak seimbang (lebih banyak pasien Tidak Diabetes), digunakan parameter *class_weight* agar model tidak bias terhadap kelas mayoritas. Proses training juga menggunakan EarlyStopping untuk menghentikan pelatihan otomatis saat model mulai overfitting.

```

from sklearn.utils.class_weight import compute_class_weight

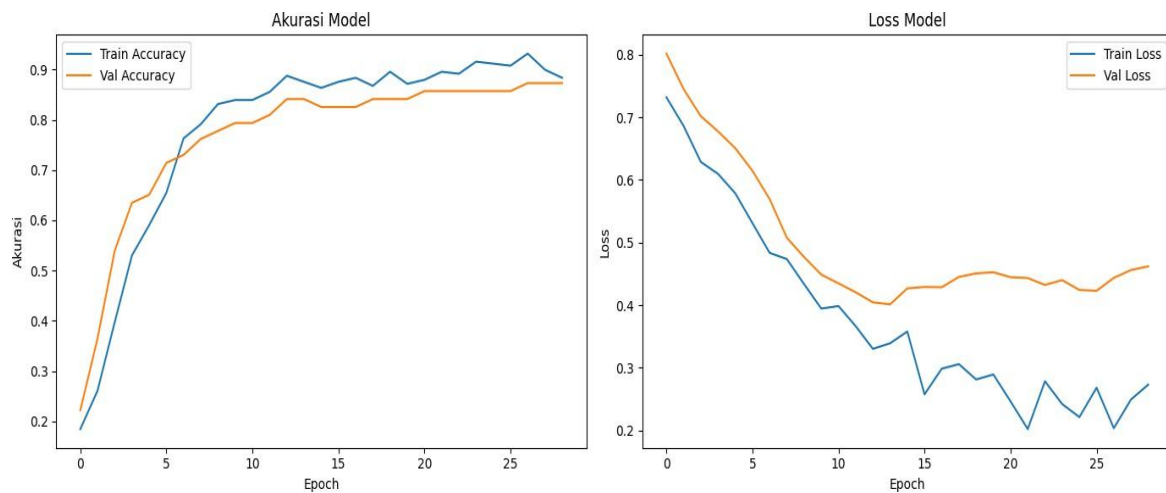
classes = np.unique(y_train)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)
class_weight = dict(zip(classes, weights))
print("Class weight:", class_weight)

early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=15, restore_best_weights=True
)

history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=150,
    batch_size=16,
    class_weight=class_weight,
    callbacks=[early_stop],
    verbose=1
)

```

Kurva akurasi dan loss selama proses training dipantau untuk memastikan model belajar dengan baik tanpa overfitting berlebihan.



Gambar 4. Kurva akurasi (kin) dan loss (kanan) model selama proses pelatihan, dibandingkan antara data latih dan data validasi.

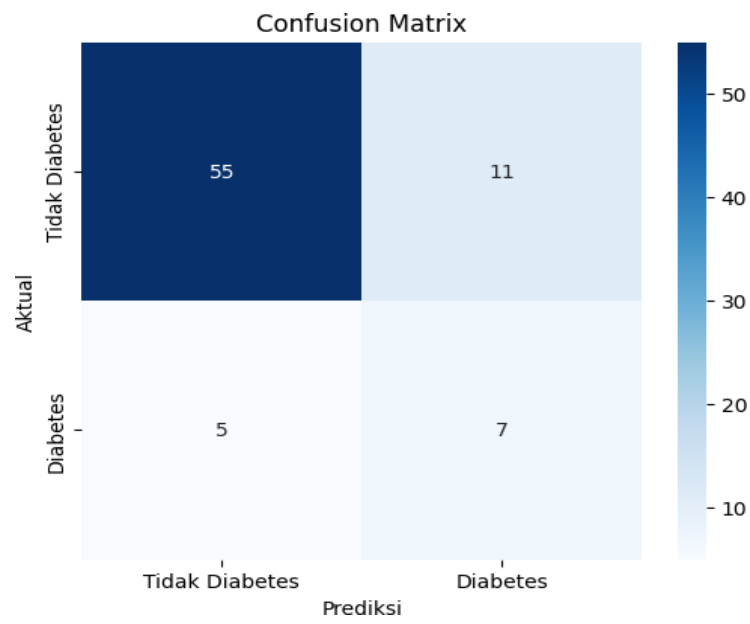
2.6 Evaluasi Model

Model dievaluasi menggunakan data uji (data yang tidak dilihat model selama training) untuk mengukur performa sebenarnya.

```
loss, accuracy = model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"Akurasi pada data uji: {accuracy:.4f}")
print(f"Loss pada data uji : {loss:.4f}")

Akurasi pada data uji: 0.7949
Loss pada data uji : 0.3867
```

Untuk melihat performa per kelas (bukan hanya akurasi keseluruhan), digunakan classification report yang menampilkan precision, recall, dan f1-score.



Gambar 5. Confusion matrix hasil prediksi model pada data uji.

3. Ringkasan Hasil

Metrik	Nilai
Akurasi pada data uji	79,49%
Loss pada data uji	0,3867
Precision (Diabetes)	0,39
Recall (Diabetes)	0,58
F1-score (Diabetes)	0,47
Precision (Tidak Diabetes)	0,92
Recall (Tidak Diabetes)	0,83
F1-score (Tidak Diabetes)	0,87

Model mencapai akurasi keseluruhan sebesar **79,49%** pada data uji. Namun, karena distribusi kelas target tidak seimbang (330 pasien Tidak Diabetes berbanding 60 pasien Diabetes), akurasi keseluruhan saja tidak cukup merepresentasikan performa model. Recall untuk kelas Diabetes sebesar 0,58 menunjukkan model mampu mendeteksi lebih dari setengah kasus diabetes yang sebenarnya, sementara precision yang lebih rendah (0,39) menunjukkan masih ada beberapa prediksi positif palsu (false positive).

4. Kesimpulan

- Model Deep Learning (ANN) berhasil dibangun dan dilatih untuk mengklasifikasikan risiko diabetes pasien berdasarkan data medis dan fisik tabular.
- Penerapan `class_weight` terbukti membantu model dalam mengenali kelas minoritas (Diabetes) yang jumlahnya jauh lebih sedikit dibanding kelas mayoritas.
- Precision dan recall pada kelas Diabetes masih dapat ditingkatkan, misalnya dengan teknik oversampling (SMOTE), penyesuaian threshold klasifikasi, atau tuning hyperparameter lebih lanjut.
- Model ini dapat menjadi dasar untuk pengembangan sistem skrining awal risiko diabetes, namun tetap memerlukan validasi klinis lebih lanjut sebelum digunakan dalam pengambilan keputusan medis nyata.